A **Stream API** (introduzida no Java 8) é um conjunto de classes e interfaces que permite processar sequências de elementos (como coleções de dados) de forma declarativa, funcional e encadeada. [1, 2]

Em vez de escrever laços de repetição tradicionais (como `for` ou `while`), você define **o que** deve ser feito com os dados, e a API cuida do controle de fluxo por trás dos panos. [1, 2]

Principais Características

- **Estilo Declarativo:** O código fica mais limpo, legível e com menos linhas em comparação aos loops imperativos.
- **Não altera a fonte de dados:** Uma Stream não armazena os elementos nem modifica a coleção original; ela apenas consome e processa os dados em um "pipeline".
- **Avaliação Preguiçosa (*Lazy Evaluation*):** As operações intermediárias só são executadas de fato quando uma **operação terminal** é acionada.
- **Suporte a Paralelismo:** É possível processar grandes volumes de dados usando múltiplos núcleos de CPU de forma simples através do `.parallelStream()`. [1, 2, 3, 4, 5, 6]

Estrutura de uma Stream

O fluxo de trabalho com uma Stream é dividido em três etapas:

1. **Fonte (*Source*):** De onde os dados vêm (geralmente uma `List`, `Set`, `Map` ou array). [1]
2. **Operações Intermediárias:** Transformam ou filtram a Stream e **retornam outra Stream**, permitindo o encadeamento (*method chaining*).
   - *Exemplos:* `filter()`, `map()`, `sorted()`, `distinct()`. [1, 2, 3]
3. **Operação Terminal:** Encerra o fluxo e retorna um resultado final (uma nova coleção, um único valor ou `void`).
   - *Exemplos:* `collect()`, `forEach()`, `count()`, `reduce()`. [1]

Exemplo Prático

Comparação entre o código tradicional (*imperativo*) e o uso da Stream API (*funcional*):

Com `for` tradicional:

java

```java
List<String> nomes = Arrays.asList("Maria", "João", "Pedro", "Ana");
List<String> resultado = new ArrayList<>();

for (String nome : nomes) {
    if (nome.startsWith("A")) {
        resultado.add(nome.toUpperCase());
    }
}
```

Use o código com cuidado.

Com a Stream API:

java

```java
List<String> nomes = Arrays.asList("Maria", "João", "Pedro", "Ana");

List<String> resultado = nomes.stream()
    .filter(n -> n.startsWith("A"))
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

Use o código com cuidado.

Para se aprofundar no tema, você pode conferir o Tutorial da API de Streams do Baeldung ou o artigo explicativo no Medium sobre Java Streams. [1, 2]

Se quiser, posso te mostrar:

- Como usar **operações de redução** (`reduce`)
- Exemplos com **processamento paralelo** (`parallelStream`)
- Ou como criar Streams a partir de **arquivos ou números**